

4 Linux文件操作

4.1 基于文件指针的文件操作

4.1.1 Linux的文件

- Linux中对目录和设备的操作都是文件操作，文件分为普通文件，目录文件，链接文件和设备文件
- 普通文件：也称磁盘文件，并且能够进行随机的数据存储(能够自由seek定位到某一个位置)
- 管道：是一个从一端发送数据，另一端接收数据的数据通道
- 目录：也称为目录文件，它包含了保存在目录中文件列表的简单文件
- 设备：该类型的文件提供了大多数物理设备的接口。它又分为两种类型：字符型设备和块设备。字符型设备一次只能读出和写入一个字节的数据，包括调制解调器、终端、打印机、声卡以及鼠标；块设备必须以一定大小的块来读出或者写入数据，块设备包括CD-ROM、RAM驱动器和磁盘驱动器等。一般而言，**字符设备用于传输数据，块设备用于存储数据**
- 链接：类似于Windows的快捷方式，指包含到达另一个文件路径的文件。
- 基于文件指针的文件操作函数是ANSI标准函数库的一部分。

"r" 只读打开

"r+" 读写打开

"w" 只写创建

"w+" 读写创建

"a" 只写追加

"a+" 读写追加

4.1.2 文件的创建，打开与关闭

- 使用文件指针来访问文件的方法是由标准C规定的，相关函数的原型为：

```
#include <stdio.h> //头文件包含
FILE* fopen(const char* path, const char* mode); //文件名 模式
int fclose(FILE* stream);
```

- fopen以mode的方式打开或创建文件，如果成功，将返回一个文件指针，失败则返回NULL。
fopen创建的文件的访问权限将以0666与当前的umask结合来确定。
- mode的可选模式列表，如下所示：

模式	读	写	位置	截断原内容	创建
rb	Y	N	文件头	N	N
rb+	Y	Y	文件头	N	N
wb	N	Y	文件头	Y	Y
wb+	Y	Y	文件头	Y	Y
ab	N	Y	文件尾	N	Y
ab+	Y	Y	文件尾	N	Y

- 在Linux系统中,mode里面的'b'(二进制)可以去掉，但是为了保持与其他系统的兼容性，建议不要去掉
- ab和ab+为追加模式，在此两种模式下，在一开始的时候读取文件内容是从文件起始处开始读取的，而无论文件读写点定位到何处，在写数据时都将是文件末尾添加（写完以后读写点就移动到文件末尾了），所以比较适合于多进程写同一个文件的情况下保证数据的完整性

4.1.3 读写文件

基于文件指针的数据读写函数较多，可分为如下几组：

数据块读写：

```
#include <stdio.h>
size_t fread(void *ptr, size_t size, size_t nmem, FILE *stream);
size_t fwrite(void *ptr, size_t size, size_t nmem, FILE *stream);
```

- fread从文件流stream 中读取nmem个元素，写到ptr指向的内存中，每个元素的大小为size个字节
- fwrite从ptr指向的内存中读取nmem个元素，写到文件流stream中，每个元素size个字节
- 所有的文件读写函数都从文件的当前读写点开始读写，读写完以后，当前读写点自动往后移动size*nmem个字节。整块copy，速度较快，但是是二进制操作

格式化读写：

```
#include <stdio.h>
int printf(const char *format, ...);
//相当于fprintf(stdout,format,...);
int scanf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
int fscanf(FILE *stream, const char *format, ...);
int sprintf(char *str, const char *format, ...);
//eg:sprintf(buf,"the string is;%s",str);
int sscanf(char *str, const char *format, ...);
```

- fprintf将格式化后的字符串写入到文件流stream中
- sprintf将格式化后的字符串写入到字符串str中

单个字符读写：

- 使用下列函数可以一次读写一个字符

```
#include <stdio.h>
int fgetc(FILE *stream);
int fputc(int c, FILE *stream);
int getc(FILE *stream);//等同于 fgetc(FILE* stream)
int putc(int c, FILE *stream);//等同于 fputc(int c, FILE* stream)
int getchar(void);//等同于 fgetc(stdin);
int putchar(int c);//等同于 fputc(int c, stdout);
```

- getchar和putc从标准输入输出流中读写数据，其他函数从文件流stream中读写数据

字符串读写：

```
char *fgets(char *s, int size, FILE *stream);
int fputs(const char *s, FILE *stream);
int puts(const char *s);//等同于 fputs(const char *s,stdout);
char *gets(char *s);//等同于 fgets(const char *s, int size, stdin);
```

- fgets和fputs从文件流stream中读写一行数据;
- puts和gets从标准输入输出流中读写一行数据。

- fgets可以指定目标缓冲区的大小，所以相对于gets安全，但是fgets调用时，如果文件中当前行的字符个数大于size，则下一次fgets调用时，将继续读取该行剩下的字符，fgets读取一行字符时，保留行尾的换行符。
- fputs不会在行尾自动添加换行符，但是puts会在标准输出流中自动添加一换行符。

文件定位：

- 文件定位指读取或设置文件当前读写点，所有的通过文件指针读写数据的函数，都是从文件的当前读写点读写数据的。常用的函数有：

```
#include <stdio.h>
int feof(FILE * stream);
//通常的用法为while(!feof(fp))，没什么太多用处
int fseek(FILE *stream, long offset, int whence);
//设置当前读写点到偏移whence 长度为offset处
long ftell(FILE *stream);
//用来获得文件流当前的读写位置
void rewind(FILE *stream);
//把文件流的读写位置移至文件开头 fseek(fp, 0, SEEK_SET);
```

- fseek设置当前读写点到偏移whence 长度为offset处，whence可以是：SEEK_SET (文件开头)、SEEK_CUR (文件当前位置)、SEEK_END (文件末尾)
- ftell获取当前的读写点
- rewind将文件当前读写点移动到文件头

4.1.4 文件的权限

```
#include <sys/stat.h>
int chmod(const char* path, mode_t mode);
//mode形如：0777 是一个八进制整型
//path参数指定的文件被修改为具有mode参数给出的访问权限。
```

4.2 目录操作

4.2.1 获取、改变当前目录

```
#include <unistd.h> //头文件
char *getcwd(char *buf, size_t size); //获取当前目录，相当于pwd命令
int chdir(const char *path); //修改当前目录，即切换目录，相当于cd命令
```

- getcwd()函数将当前的工作目录绝对路径复制到参数buf所指的内存空间，参数size为buf的空间大小。因此在调用此函数时，buf所指的内存空间要足够大，若工作目录绝对路径的字符串长度超过参数size大小，则返回值NULL，errno的值则为ERANGE
- 倘若参数buf为NULL，getcwd()会依参数size的大小自动配置内存(使用malloc())，如果参数size也为0，则getcwd()会依工作目录绝对路径的字符串长度来决定所配置的内存大小，进程可以在使用完此字符串后自动利用free()来释放此空间。所以常用的形式：

```
getcwd(NULL, 0);
```

- chdir()函数：用来将当前的工作目录改变成以参数path所指的目录

```
#include<unistd.h>
int main()
{
    chdir("/tmp");
    printf("current working directory: %s\n",getcwd(NULL,0));
}
```

4.2.2 创建和删除目录

```
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
int mkdir(const char *pathname, mode_t mode); //创建目录,mode是目录权限
int rmdir(const char *pathname); //删除目录
```

- 如果需要查看库函数mkdir而不是shell命令mkdir，使用man命令的时候需要指定系统库函数帮助手册（也就是mkdir(2)）
- 如果函数的（指针）参数使用const来进行修饰，这个参数就称为传入参数。这意味在函数的内部是不能够通过指针来修改传入参数的内容

\$ man 2 mkdir

- 查看和修改环境变量

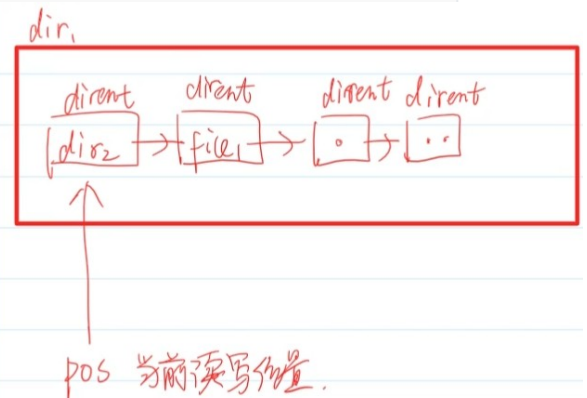
查看环境变量

\$ echo \$PATH

修改环境（系统路径）变量（只对本次生效）

\$ export PATH=\$PATH:新目录

Dir 目录流



4.2.3 目录的存储原理

- 对于任何文件，为了快速定位文件在磁盘当中的位置，文件系统在设计的时候就需要利用专门的索引结构来管理所有文件。索引结构的基本单位是索引结点，每个索引结点的具有固定的大小，里面存放了单个文件的位置、文件类型、权限、修改时间等等信息。文件系统将所有索引结点用数组的形式组织起来，并利用一个辅助的位图来实现高效管理文件信息
- 在Linux当中，目录是一种特别的文件，它的总体大小固定。它的数据块当中把很多文件的文件名和索引结点存放在一起。因为文件的文件名大小不一，为了避免磁盘碎片和支持频繁增加修改，所以目录采用链式结构来存储来组织各种文件的信息。链式结构的结点就是dirent结点，它的定义如下。可以看出，要访问下个dirent结点，实际是依赖于本结点中d_off属性

```
struct dirent{
    ino_t d_ino; //该文件的inode，即文件的索引地址
    off_t d_off; //到下一个dirent的偏移，next 指针
    unsigned short d_reclen; //文件名长度
    unsigned char d_type; //所指的文件类型
    char d_name[256]; //文件名
};
```

4.2.4 目录相关操作

```
#include <sys/types.h>
#include <dirent.h>
DIR *opendir(const char *name);    //打开一个目录
struct dirent *readdir(DIR *dir);  //读取目录的一项信息，并返回该项信息的结构体指针
void rewinddir(DIR *dir);          //重新定位到目录文件的头部
void seekdir(DIR *dir, off_t offset); //用来设置目录流目前的读取位置
off_t telldir(DIR *dir);           //返回目录流当前的读取位置
int closedir(DIR *dir);            //关闭目录文件

#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
int stat(const char *pathname, struct stat *buf); //获取文件状态
```

- 读取目录信息的步骤为：

1. 用opendir函数打开目录，获得DIR指针。DIR称为目录流，类似于标准输入输出，每次使用readdir以后，它会将位置移动到下一个文件
2. 使用readdir函数迭代读取目录的内容
3. 用closedir函数关闭目录

- dirent当中采用了类似于变长数组的形式来存放文件名，但是会提供一些冗余的空间，这样当调整文件名的时候，如果新文件名的长度不超过原来分配的空间，则不需要调整分配的空间
- 类似于地址和内存的关系，inode（索引结点）描述了文件在磁盘上的具体位置信息。在ls命令中添加-i参数可查看文件的inode信息。那么所谓的硬链接，就是指inode相同的文件。一个inode的结点上的硬链接的个数就称为引用计数

```
$ ls -ial
```

查看所有文件的inode信息

```
$ ln 当前文件 目标
```

建立名为“目标”的硬链接

- 这里可以检查. 文件引用计数，发现是2。这是因为一个目录可以通过本目录或者是父目录来访问它本身，若它还有子目录，它的引用数也会增加
- 删除磁盘上文件的时候，只有引用计数为0时候，才会将磁盘内容移除文件系统（断开和目录的链接）
- 当然，为了避免引用死锁，一般用户是不能使用ln命令来为目录建立硬链接。

//使用深度优先遍历访问目录的例子

```
#include <func.h>
int printDir(char *path)
{
    DIR* pdir = opendir(path);
    ERROR_CHECK(pdir, NULL, "opendir");
    struct dirent *pdirent;
    char buf[1024]; //注意递归时传递的路径是否合理
    while((pdirent = readdir(pdir)))
    {
        if(strcmp(pdirent->d_name, ".") == 0 || strcmp(pdirent->d_name, "..") ==
0)
        {
            continue;
        }
    }
```

```

        printf("%s\n",pdirent->d_name);
        sprintf(buf,"%s%s",path,"/",pdirent->d_name);//这里不需要担心斜杠太多的问题
        if(pdirent->d_type == 4)
        {
            printDir(buf);
        }
    }
    closedir(pdir);
    return 0;
}

int tabPrintDir(char *path,int width)//这里实现了类似tree的效果
{
    DIR* pdir = opendir(path);
    ERROR_CHECK(pdir,NULL,"opendir");
    struct dirent *pdirent;
    char buf[1024];
    while((pdirent = readdir(pdir)))
    {
        if(strcmp(pdirent->d_name,".") == 0 || strcmp(pdirent->d_name,"..") ==
0)
        {
            continue;
        }
        printf("%*s\n",width,"",pdirent->d_name);
        sprintf(buf,"%s%s",path,"/",pdirent->d_name);
        if(pdirent->d_type == 4)
        {
            tabPrintDir(buf,width+4);
        }
    }
    closedir(pdir);
    return 0;
}

int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    puts(argv[1]);
    printDir(argv[1]);
    tabPrintDir(argv[1],0);
    return 0;
}

```

- seekdir()函数用来设置目录流目前的读取位置，再调用readdir()函数时，便可以从此新位置开始读取。参数offset代表距离目录文件开头的偏移量
- 使用readdir()时，如果已经读取到目录末尾，又想重新开始读，则可以使用rewinddir函数将文件指针重新定位到目录文件的起始位置
- telldir()函数用来返回目录流当前的读取位置

```

//下面是一个例子
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    DIR* pdir = opendir(argv[1]);
    ERROR_CHECK(pdir,NULL,"opendir");
    struct dirent *pdirent;

```

```

off_t pos;
while((pdirent = readdir(pdir)))
{
    printf("ino = %ld len = %d type = %d filename = %s\n",pdirent-
>d_ino,pdirent->d_reclen,pdirent->d_type,pdirent->d_name);
    if(strcmp(pdirent->d_name,"a.out") == 0)
    {
        pos = telldir(pdir);//只会成功，不会失败
    }
}
//seekdir(pdir,pos);
rewinddir(pdir);
pdirent = readdir(pdir);
printf("-----\n");
printf("ino = %ld len = %d type = %d filename = %s\n",pdirent-
>d_ino,pdirent->d_reclen,pdirent->d_type,pdirent->d_name);
return 0;
}

```

- 使用stat结构体（又称为inode信息）可以获取文件信息

```

//结构体stat的定义
struct stat {
    dev_t      st_dev;    /*如果是设备，返回设备表述符，否则为0*/
    ino_t      st_ino;    /* i节点号 */
    mode_t     st_mode;   /* 文件类型 */
    nlink_t    st_nlink;  /* 链接数 */
    uid_t      st_uid;    /* 属主ID */
    gid_t      st_gid;    /* 组ID */
    dev_t      st_rdev;   /* 设备类型*/
    off_t      st_size;   /* 文件大小，字节表示 */
    blksize_t  st_blksize; /* 块大小*/
    blkcnt_t   st_blocks; /* 块数 */
    time_t     st_atime;  /* 最后访问时间*/
    time_t     st_mtime;  /* 最后修改时间*/
    time_t     st_ctime;  /* 最后权限修改时间 */
};

```

- 使用man命令可以查看stat的内容

```
$ man 2 stat
```

```

//示例
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int ret;
    struct stat buf;
    ret = stat(argv[1],&buf);
    ERROR_CHECK(ret,-1,"stat");

    printf("%x %ld %s %s %ld
%s\n",buf.st_mode,buf.st_nlink,getpwuid(buf.st_uid)-
>pw_name,getgrgid(buf.st_gid)->gr_name,buf.st_size,ctime(&buf.st_mtime));
    return 0;
}

```



```
}
```

- 注意getpwuid和getgrgid需要包含头文件pwd.h和grp.h

4.3 基于文件描述符的文件操作

4.3.1 文件描述符

- 上一节所讨论的文件都是使用FILE类型来管理，根据FILE类型的结构可知它的本质是一个缓冲区。与FILE类型相关的文件操作（比如fopen, fread等等）称为带缓冲的IO，它们是ISO C的组成部分
- POSIX标准支持另一类不带缓冲区的IO。在这里，文件是使用文件描述符来描述。文件描述符是一个非负整数。从原理上来说，进程地址空间的内核部分里面会维护一个已经打开的文件的数组，这个数组用来管理所有已经打开的文件（打开的文件存储在进程地址空间的内核区中，称为文件对象），而文件描述符就是这个数组的索引。因此，文件描述符可以实现进程和打开文件之间的交互

4.3.2 打开、创建和关闭文件

- 使用open函数可以打开或者创建并打开一个文件，使用creat函数可以创建一个文件

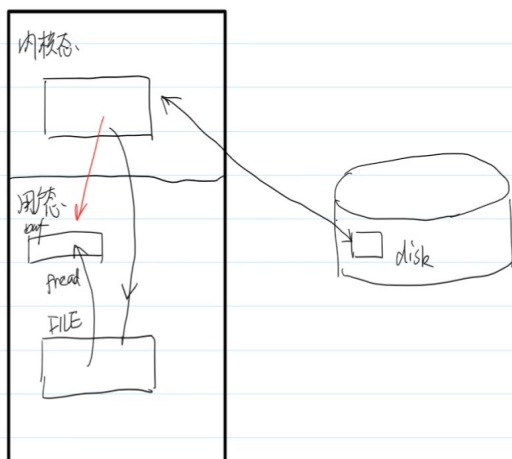
```
#include <sys/types.h> //头文件
#include <sys/stat.h>
#include <fcntl.h>
int open(const char *pathname, int flags); //文件名 打开方式
int open(const char *pathname, int flags, mode_t mode); //文件名 打开方式 权限
int creat(const char *pathname, mode_t mode); //文件名 权限
//creat现在已经不常用了，它等价于
open(pathname, O_CREAT | O_TRUNC | O_WRONLY, mode);
```

- 执行成功时，open函数返回一个文件描述符，表示已经打开的文件；执行失败是，open函数返回-1，并设置相应的errno
- flags和mode都是一组掩码的合成值，flags表示打开或创建的方式，mode表示文件的访问权限。
- flags的可选项有

不带缓冲的文件IO *unbuffered IO*

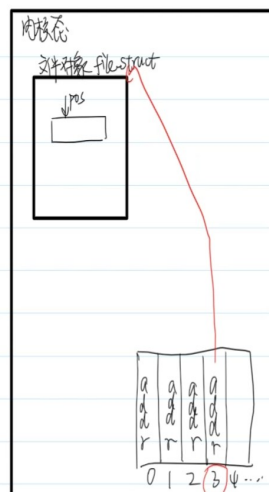
2024年1月23日 17:45

没有用户态缓冲区



内核如何管理数据结构

2024年1月23日 17:55



内核管理打开的文件以当作文件对象

文件描述符

		fopen() mode	open() flags
掩码	含义	<u>r</u>	O_RDONLY
O_RDONLY	以只读的方式打开	<u>w</u>	O_WRONLY O_CREAT O_TRUNC
O_WRONLY	以只写的方式打开	<u>a</u>	O_WRONLY O_CREAT O_APPEND
O_RDWR	以读写的方式打开	<u>r+</u>	O_RDWR
O_CREAT	如果文件不存在，则创建文件	<u>w+</u>	O_RDWR O_CREAT O_TRUNC
O_EXCL	仅与O_CREAT连用，如果文件已存在，则open失败	<u>a+</u>	O_RDWR O_CREAT O_APPEND
O_TRUNC	如果文件存在，将文件的长度截至0		
O_APPEND	已追加的方式打开文件，每次调用write时，文件指针自动先移到文件尾，用于多进程写同一个文件的情况。		
O_NONBLOCK O_NDELAY	对管道、设备文件和socket使用，以非阻塞方式打开文件，无论有无数据读取或等待，都会立即返回进程之中		
O_SYNC	同步打开文件，只有在数据被真正写入物理设备后才返回		
O_ASYNC	对管道、设备文件和socket使用，开启信号驱动IO。一旦可读或者可写发送信号		

- 使用完文件以后，要记得使用close来关闭文件。一旦调用close，则该进程对文件所加的锁全都被释放，并且使文件的打开引用计数减1，只有文件的打开引用计数变为0以后，文件才会被真正的关闭。用ulimit -a命令可以查看单个进程能同时打开的文件的上限

```
int close(int fd); //fd表示文件描述词,是先前由open或creat创建文件时的返回值。
```

- mode通常采用直接赋数值的形式，例如：

```
int fd = open("file", O_RDWR | O_CREAT, 0755); //表示给755的权限
if(-1 == fd)
{
    perror("open failed!\n");
    exit(-1);
}
```

4.3.3 读写文件

- 使用read和write来读写文件，它们统称为不带缓冲的IO

```
#include <unistd.h>
ssize_t read(int fd, void *buf, size_t count); //文件描述符 缓冲区 长度
ssize_t write(int fd, const void *buf, size_t count);
```

- 对于read和write函数，出错返回-1，读取完了之后，返回0，其他情况返回读写的个数。

```
//读取文件内容
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 2);
```

```

    int fd = open(argv[1], O_RDWR);
    ERROR_CHECK(fd, -1, "open");
    printf("fd = %d\n", fd);
    char buf[128] = {0};
    int ret = read(fd, buf, sizeof(buf));
    printf("buf = %s, ret = %d\n", buf, ret);
    close(fd);
    return 0;
}

//写入文件描述符
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 2);
    int fd = open(argv[1], O_RDWR);
    ERROR_CHECK(fd, -1, "open");
    printf("fd = %d\n", fd);
    int ret = write(fd, &fd, sizeof(fd));
    printf("ret = %d\n", ret);
    close(fd);
    return 0;
}

//如果希望查看文件的2进制信息，在vim中输入命令:%!xxd
//使用od -h 命令也可查看文件的16进制形式

```

read 的效率问题：

- 使用不带缓冲 IO 的时候，CPU 需要陷入内核态来处理文件读取。如果频繁地使用 read 来读取少量数据，数据的读取效率会比较低，read 读磁盘文件是非阻塞，读设备文件是阻塞的。

使用read的场景：

- 读取常规文件时，文件内容大于读取长度（即还没有遇到 EOF，读取字符数达到 count），此时返回值等于 count
- 读取常规文件时，文件内容小于读取长度，此时返回值等于文件内容长度
- 读取网络文件的时候，由于数据传输不稳定，可能会导致文件还没有传输完成，read 函数就已经返回的情况。此时返回值等于成功读取的字符数

4.3.4 改变文件大小

- 使用ftruncate函数可以文件大小

```

#include <unistd.h>
int ftruncate(int fd, off_t length);

```

- 函数ftruncate会将参数fd指定的文件大小改为参数length指定的大小。参数fd为已打开的文件描述词，而且必须是以**写入模式**打开的文件。如果原来的文件大小比参数length大，则超过的部分会被删去（实际上修改了文件的inode信息）。执行成功则返回0，失败返回-1

实例：

```
//示例
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fd = open(argv[1],O_WRONLY);
    ERROR_CHECK(fd,-1,"open");
    printf("fd = %d\n",fd);
    int ret = ftruncate(fd,3);
    ERROR_CHECK(ret,-1,"ftruncate");
    return 0;
}
```

- 使用mmap函数经常配合函数ftruncate来扩大文件大小

```
//示例
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fd = open(argv[1],O_RDWR);
    ERROR_CHECK(fd,-1,"open");
    printf("fd = %d\n",fd);
    ftruncate(fd,5)
    char *p;
    p = (char *)mmap(NULL,5,PROT_READ|PROT_WRITE,MAP_SHARED,fd,0);
    ERROR_CHECK(p,(char *)-1,"mmap");
    p[5] = 0;
    printf("%s\n",p);
    p[0] = 'H';
    munmap(p,5);
    close(fd);
    return 0;
}
```

4.3.5 文件映射

- 使用mmap接口可以实现直接将一个磁盘文件映射到存储空间的一个缓冲区上面，无需使用read和write进行IO

```
#include <sys/mman.h>
void *mmap(void *adr, size_t len, int prot, int flag, int fd, off_t off);
```

- addr参数用于指定映射存储区的起始地址。这里设置为NULL，这样就由系统自动分配（通常是在堆空间里面寻找）。fd参数是一个文件描述符，使用时必须要已经打开。prot参数用来表示权限。PROT_READ,PROT_WRITE表示可读可写。flag参数在目前是采用MAP_SHARED，后面讲解进程通信的时候会介绍其他类型
- 为什么mmap需要和ftruncate联合使用？因为分配的缓冲区的大小和偏移量的大小是有限制的，它必须是虚拟内存页大小的整数倍。如果文件大小较小，那么超过文件大小返回的缓冲区操作将不会修改文件；如果文件大小为0，还会出现Bus error（小文件建立了大映射）；使用完毕后，用munmap 来释放 mmap 开辟的空间。

4.3.6 文件定位

- 函数lseek将文件指针设定到相对于whence，偏移值为offset的位置。它的返回值是读写点距离文件开始的距离

```
#include <sys/types.h>
#include <unistd.h>
off_t lseek(int fd, off_t offset, int whence); //fd文件描述词
//whence 可以是下面三个常量的一个
//SEEK_SET 从文件头开始计算
//SEEK_CUR 从当前指针开始计算
//SEEK_END 从文件尾开始计算
```

```
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 2);
    int fd = open(argv[1], O_RDWR);
    ERROR_CHECK(fd, -1, "open");
    int ret = lseek(fd, 5, SEEK_SET);
    printf("pos = %d\n", ret);
    char buf[128] = {0};
    read(fd, buf, sizeof(buf));
    printf("buf = %s\n", buf);
    close(fd);
    return 0;
}
```

- 利用该函数可以实现文件空洞：对一个新建的空文件，可以定位到偏移文件开头1024个字节的地方，在写入一个字符，则相当于给该文件分配了1025个字节的空间，形成文件空洞。通常用于多进程间通信的时候的共享内存。（在某些文件系统的实现中，这些空洞甚至不会占用磁盘空间）

```
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 2);
    int fd = open(argv[1], O_RDWR);
    ERROR_CHECK(fd, -1, "open");
    int ret = lseek(fd, 1024, SEEK_SET);
    write(fd, "a", 1);
    close(fd);
    return 0;
}
```

4.3.7 获取文件信息

- 可以通过fstat和stat函数获取文件信息，调用完毕后，文件信息被填充到结构体struct stat变量中，函数原型为：

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
int stat(const char *file_name, struct stat *buf); //文件名 stat结构体指针
int fstat(int fd, struct stat *buf); //文件描述词 stat结构体指针
```

- 对于结构体的成员st_mode，有一组宏可以进行文件类型的判断

宏	描述
S_ISLNK(mode)	判断是否是符号链接
S_ISREG(mode)	判断是否是普通文件
S_ISDIR(mode)	判断是否是目录
S_ISCHR(mode)	判断是否是字符型设备
S_ISBLK(mode)	判断是否是块设备
S_ISFIFO(mode)	判断是否是命名管道
S_ISSOCK(mode)	判断是否是套接字

```
//获得文件的大小
#include<sys/stat.h>
#include<unistd.h>
int main(int argc, char *argv[])
{
    struct stat buf;
    stat (argv[1],&buf);
    printf("file size = %d\n",buf.st_size);//st_size可以得到文件大小
}
//如果用fstat函数实现，如下：
//int fd = open ("/etc/passwd",O_RDONLY); //先获得文件描述词
//fstat(fd, &buf);
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fd = open(argv[1],O_RDWR|__O_PATH);

    ERROR_CHECK(fd,-1,"open");
    struct stat buf;
    int ret = fstat(fd, &buf);
    ERROR_CHECK(ret, -1, "fstat");
    if(S_ISDIR(buf.st_mode))
    {
        printf("directory\n");
    }
    else if(S_ISREG(buf.st_mode))
    {
        printf("regular file\n");
    }
    else if(S_ISLNK(buf.st_mode))
    {
        printf("link file\n");
    }
    printf("the size of file is: %ld\n",buf.st_size);
    close(fd);
    return 0;
}
```

4.3.8 文件描述符的复制

- 系统调用函数dup和dup2可以实现文件描述符的复制
- dup返回一个新的文件描述符（是自动分配的，数值是没有使用的文件描述符的最小编号）。这个新的描述符是旧文件描述符的拷贝。这意味着两个描述符共享同一个数据结构
- dup2允许调用者用一个有效描述符(oldfd)和目标描述符(newfd)。函数成功返回时，目标描述符将变成旧描述符的复制品，此时两个文件描述符现在都指向同一个文件，并且是函数第一个参数（也就是oldfd）指向的文件（执行完成以后，如果newfd已经打开了文件，该文件将会被关闭）
原型为：

```
#include <unistd.h>
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

- 文件描述符的复制是指用另外一个文件描述符指向同一个打开的文件，它完全不同于直接给文件描述符变量赋值，下面是文件描述符赋值的例子：

```
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fd = open(argv[1],O_RDWR);
    ERROR_CHECK(fd,-1,"open");
    printf("fd = %d\n",fd);
    int fd1 = fd;
    close(fd);
    char buf[128] = {0};
    int ret = read(fd1, buf, sizeof(buf));
    ERROR_CHECK(ret, -1, "read");
    return 0;
}
```

- 在此情况下，两个文件描述符变量的值相同，指向同一个打开的文件，但是内核的文件打开引用计数还是为1，所以close(fd)或者close(fd1)都会导致文件立即关闭
- 如果使用文件描述符的复制，则情况有所区别

```
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fd = open(argv[1],O_RDWR);
    ERROR_CHECK(fd,-1,"open");
    printf("fd = %d\n",fd);
    int fd1 = dup(fd);
    close(fd);
    char buf[128] = {0};
    int ret = read(fd1, buf, sizeof(buf));
    ERROR_CHECK(ret, -1, "read");
    printf("buf = %s\n", buf);
    return 0;
}
```

- dup的原理：当使用文件的时候，为了和硬件（比如磁盘）建立联系，进程地址空间中应当分配一片空间存放各个已经打开文件的inode信息（此时的文件信息已经放在内存，和实际磁盘内容无关

了), Linux当中是采用链表的方式将它们组织起来, 称为inode表。除此以外, 系统为了高效管理文件, 需要一个额外的数据结构来管理文件, 称为文件表。文件表里面存放了文件的状态标志 (典型的比如引用计数)、偏移量以及inode表的指针。文件表和inode表是在进程地址空间里面, 是存放内核区中的, 并且这些内容是所有进程共享的 (所以多进程同时写入会有问题)

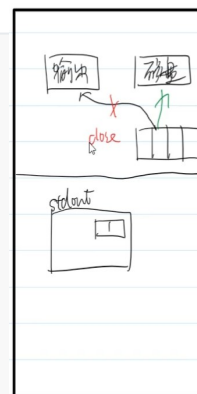
- 而我们所使用的文件描述符与内核区中的另一个数据结构文件指针表有关, 文件指针表的索引就是描述符, 而数组的内容就是文件表项的指针。当执行dup函数以后, 在文件指针表当中, 会有两个不同的描述符来描述同一个文件, 而在文件表当中, 该文件的引用计数会自增1。当关闭文件时, 文件指针表会移除该文件相关的项, 并且文件表中该文件的引用计数会自减1, 当引用计数为0的时候, 文件表以及inode表的表项会被释放
- 使用dup函数可以实现输出重定向

```
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fd = open(argv[1],O_RDWR);
    ERROR_CHECK(fd,-1,"open");
    printf("\n");
    close(STDOUT_FILENO);
    int fd1 = dup(fd);
    printf("fd1 = %d\n", fd1);
    close(fd);
    printf("the out of stdout\n");
    return 0;
}
```

- 该程序首先打开了一个文件, 返回一个文件描述符, 因为默认的就打开了0,1,2表示标准输入, 标准输出, 标准错误输出。而用close(STDOUT_FILENO);则表示关闭标准输出, 此时文件描述符1就空着然后dup(fd);则会复制一个文件描述符到当前未打开的最小描述符, 此时这个描述符为1。后面关闭fd自身, 然后在用标准输出的时候, 发现标准输出重定向到你指定的文件了。那么printf所输出的内容也就直接输出到文件 (因为printf的原理就是将内容输入到描述符为1的文件里面)
- dup2(int fdold,int fdnew)也是进行描述符的复制, 只不过采用此种复制, 新的描述符由用户用参数fdnew显式指定。对于dup2, 如果fdnew已经指向一个已经打开的文件, 内核会首先关闭掉fdnew所指向的原来的文件。此时再针对于fdnew文件描述符操作的文件, 则采用的是fdold的文件描述符。如果成功dup2的返回值于fdnew相同, 否则为-1. 重定向

```
//使用输出重定向
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fd = open(argv[1],O_RDWR);
    ERROR_CHECK(fd,-1,"open");
    printf("\n");
    int fd1 = dup2(fd,STDOUT_FILENO);
    printf("fd1 = %d\n", fd1);
    close(fd);
    printf("the out of stdout\n");
    return 0;
}
```

```
//更加复杂的例子
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
```



```
#include <54func.h>
int main(int argc, char *argv[])
{
    // ./11_redirect file1
    ARGS_CHECK(argc,2);
    printf("You can see me!\n");
    // 隐藏要求 close之前先打印一个换行
    close(STDOUT_FILENO); // STDOUT_FILENO 1
    int fd = open(argv[1],O_RDWR);
    ERROR_CHECK(fd,-1,"open");
    printf("fd = %d\n", fd);
    printf("You can't see me!\n");
    return 0;
}
```



```

int fd = open(argv[1], O_RDWR);
ERROR_CHECK(fd, -1, "open");
printf("\n");
int fd0 = 100;
dup2(STDOUT_FILENO, fd0);
int fd1 = dup2(fd, STDOUT_FILENO);
printf("fd1 = %d\n", fd1);
close(fd);
printf("the out of stdout 1\n");
dup2(fd0, STDOUT_FILENO);
printf("the out of stdout 2\n");
return 0;
}

```

- 提问：如何实现将标准输出和标准错误分别重定向到文件里面？

4.3.9 文件描述符和文件指针

- fopen函数实际在运行的过程中也获取了文件的文件描述符。使用fileno函数可以得到文件指针的文件描述符。当使用fopen获取文件指针以后，依然是可以使用文件描述符来执行IO，例如

```

#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 2);
    FILE* fp = fopen(argv[1], "rb+");
    ERROR_CHECK(fp, NULL, "fopen");
    int fd = fileno(fp);
    printf("fd = %d\n", fd);
    char buf[128] = {0};
    read(fd, buf, 5);
    printf("buf = %s\n", buf);
    //使用read接口也是能够正常读取内容的
    return 0;
}

```

- fopen的原理：fopen函数在执行的时候，会先调用open函数，打开文件并且获取文件对象的信息（通过文件描述符可以获取文件对象的具体信息），然后fopen函数会在用户态空间申请一片空间作为缓冲区
- fopen的优势：因为read和write是系统调用，需要频繁地切换用户态和内核态，所以比较耗时。借助用户态缓冲区，可以减少read和write的次数，使用fdopen函数可以根据文件描述符生成用户态缓冲区

```

#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 2);
    int fd = open(argv[1], O_RDWR);
    ERROR_CHECK(fd, -1, "open");
    FILE* fp = fdopen(fd, "rb+");
    ERROR_CHECK(fp, NULL, "fdopen");
    char buf[128] = {0};
    fread(buf, 1, sizeof(buf), fp);
    printf("buf = %s\n", buf);
    return 0;
}

```

- 如果需要高效地使用不带缓冲IO，为了和存储体系匹配，最好是一次读取/写入一个块（通常是4K）大小的数据。另外如果需要使用内存映射，也应当使用open函数来打开文件
- 如果获取了文件指针，就不要通过文件描述符的方式来关闭文件

```
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    FILE* fp = fopen(argv[1], "rb+");
    ERROR_CHECK(fp, NULL, "fopen");
    close(fileno(fp)); //如果使用fd=fileno(fp),那么close以后fd的数值不会发生改变
    char buf[128] = {0};
    char *ret = fgets(buf, sizeof(buf), fp);
    ERROR_CHECK(ret, NULL, "fgets");
    printf("buf = %s\n", buf);
    return 0;
}

//出现报错 fgets: Bad file descriptor
```

4.3.10 标准输入输出文件描述符

- 与标准的输入输出流对应，在更底层的实现是用标准输入、标准输出、标准错误文件描述符表示的。它们分别用STDIN_FILENO、STDOUT_FILENO和STDERR_FILENO三个宏表示，值分别是0、1、2三个整型数字

4.3.11 管道

- （有名）管道文件是用来数据通信的一种文件，它是半双工通信，它在ls -l命令中显示为p，它不能存储数据，不占磁盘空间

传输方式	含义
全双工	双方可以同时向另一方发送数据
半双工	某个时刻只能有一方向另一方发送数据，其他时刻的传输方向可以相反
单工	永远只能由一方向另一方发送数据

```
$ mkfifo [管道名字]
使用cat打开管道可以打开管道的读端
$ cat [管道名字]
打开另一个终端，向管道当中输入内容可以实现写入内容
$ echo "string" > [管道名字]
此时读端也会显示内容
```

- 当然也可以使用C程序来分别实现读端和写端

```
//读端
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fdr = open(argv[1], O_RDONLY);
    ERROR_CHECK(fdr, -1, "open");
```

使用系统调用操作管道

2024年1月25日 17:18

open. close read write

△ 打开管道的一端 O_RDONLY/O_WRONLY

open("pipe", O_RDONLY)

① 创建管道连接的FD

② 分配fd

③ 阻塞到对端被打

open管道会引发阻塞

```

    printf("fdr = %d\n", fdr);
    char buf[128] = {0};
    read(fdr, buf, sizeof(buf)); //读管道会引发阻塞和读设备类似，因为写端不写入数据
    printf("buf = %s\n", buf);
    return 0;
}
//写端
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 2);
    int fdw = open(argv[1], O_WRONLY);
    ERROR_CHECK(fdw, -1, "open");
    printf("fdw = %d\n", fdw);
    char buf[128] = "helloworld";
    write(fdw, buf, strlen(buf));
    printf("buf = %s\n", buf);
    return 0;
}

```

- 禁止使用vim来打开管道文件

4.4 I/O多路转接模型

4.4.1 读取操作的阻塞

- 阻塞：在目前的模式下，read函数如果不能从文件中读取内容，就将进程的状态切换到阻塞状态，不再继续执行

```

//在写端写入时添加sleep(10)
...
sleep(10);
write
...
//再次测试的时候发现读端会明显延迟

```

- 因为管道文件是半双工通信，为了实现全双工通信，可以使用2个管道文件。这样就可以实现即时聊天。若读端先 close，写端后 write，写端进程会崩溃；写端先 close，读端后 read，read 不会阻塞，会立即返回，返回值是 0。

```

//1号
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc, 3);
    int fdr = open(argv[1], O_RDONLY); //管道打开的时候，必须要先将读写端都打开之后才能继续
    int fdw = open(argv[2], O_WRONLY);
    printf("I am chat1\n");
    char buf[128] = {0};
    while(1)
    {
        memset(buf, 0, sizeof(buf));
        read(STDIN_FILENO, buf, sizeof(buf));
        write(fdw, buf, strlen(buf)-1);
        memset(buf, 0, sizeof(buf));
        read(fdr, buf, sizeof(buf));
        printf("buf = %s\n", buf);
    }
}

```

```

    }
    return 0;
}
//2号
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,3);
    int fdw = open(argv[1],O_WRONLY);//管道打开的时候，必须要先将读写端都打开之后才能继续
    int fdr = open(argv[2],O_RDONLY);
    printf("I am chat2\n");
    char buf[128] = {0};
    while(1)
    {
        memset(buf,0,sizeof(buf));
        read(fdr, buf, sizeof(buf));
        printf("buf = %s\n", buf);
        memset(buf,0,sizeof(buf));
        read(STDIN_FILENO, buf, sizeof(buf));
        write(fdw, buf, strlen(buf)-1);
    }
    return 0;
}
//这里经常会有阻塞

```

4.4.2 I/O多路转接模型和select

- I/O多路转接模型：在这种模型下，如果请求的I/O操作阻塞，且它不是真正阻塞I/O，而是让其中的一个函数等待，在这期间，I/O还能进行其他操作。如本节要介绍的select()函数，就是属于这种模型

```

#include <sys/select.h>
#include <sys/time.h>

int select(int maxfd, fd_set *readset,fd_set *writeset, fd_set *exceptionset,
struct timeval * timeout);

```

- 返回值是就绪描述字的正数目，0——超时，-1——出错

参数解释：

maxfd: 最大的文件描述符（其值应该为最大的文件描述符字 + 1）

readset: 内核读操作的描述符字集合

writeset: 内核写操作的描述符字集合

exceptionset: 内核异常操作的描述符字集合

timeout: 等待描述符就绪需要多少时间。NULL代表永远等下去，一个固定值代表等待固定时间，0代表根本不等待，检查描述字之后立即返回

```

//readset、writeset、exceptionset都是fd_set集合
//集合的相关操作如下：
void FD_ZERO(fd_set *fdset);      /* 将所有fd清零 */
void FD_SET(int fd, fd_set *fdset); /* 增加一个fd */
void FD_CLR(int fd, fd_set *fdset); /* 删除一个fd */
int FD_ISSET(int fd, fd_set *fdset); /* 判断一个fd是否有设置 */

```

- 一般来说，在使用select函数之前，首先要使用FD_ZERO和FD_SET来初始化文件描述符集，在使用select函数时，可循环使用FD_ISSET测试描述符集，在执行完对相关文件描述符之后，使用FD_CLR来清除描述符集。

```
//chat1.c
//编译后运行
//$ ./chat1 1.pipe 2.pipe
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,3);
    int fdr = open(argv[1],O_RDONLY);//管道打开的时候，必须要先将读写端都打开之后才能继续
    int fdw = open(argv[2],O_WRONLY);
    printf("I am chat1\n");
    char buf[128] = {0};
    int ret;
    fd_set rdset;
    while(1)
    {
        FD_ZERO(&rdset);
        FD_SET(STDIN_FILENO,&rdset);
        FD_SET(fdr,&rdset);
        ret = select(fdr+1, &rdset, NULL, NULL, NULL);
        if(FD_ISSET(STDIN_FILENO, &rdset))
        {
            memset(buf,0,sizeof(buf));
            read(STDIN_FILENO, buf, sizeof(buf));
            write(fdw, buf, strlen(buf)-1);
        }
        if(FD_ISSET(fdr, &rdset))
        {
            memset(buf,0,sizeof(buf));
            read(fdr, buf, sizeof(buf));
            printf("buf = %s\n", buf);
        }
    }
    return 0;
}
```

```
//chat2.c
//编译后运行(注意管道建立连接的顺序)
//$ ./chat2 1.pipe 2.pipe
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,3);
    int fdw = open(argv[1],O_WRONLY);//管道打开的时候，必须要先将读写端都打开之后才能继续
    int fdr = open(argv[2],O_RDONLY);
    printf("I am chat2\n");
    char buf[128] = {0};
    int ret;
    fd_set rdset;
    while(1)
    {
        FD_ZERO(&rdset);
        FD_SET(STDIN_FILENO,&rdset);
        FD_SET(fdr,&rdset);
```

```

    ret = select(fdr+1, &rdset, NULL, NULL, NULL);
    if(FD_ISSET(STDIN_FILENO, &rdset))
    {
        memset(buf,0,sizeof(buf));
        read(STDIN_FILENO, buf, sizeof(buf));
        write(fdw, buf, strlen(buf)-1);
    }
    if(FD_ISSET(fdr, &rdset))
    {
        memset(buf,0,sizeof(buf));
        read(fdr, buf, sizeof(buf));
        printf("buf = %s\n", buf);
    }
}
return 0;
}

```

- fdset实际上是一个文件描述符的位图，采用数组的形式来存储。下面是一个简化版本的实现方法

```

//fd_set的成员是一个长整型的结构体
typedef long int __fd_mask;
//将字节转化为位
#define __NFDBITS (8 * (int) sizeof (__fd_mask))
//位图-判断是否存在文件描述符d
#define __FD_MASK(d) ((__fd_mask) (1UL << ((d) % __NFDBITS)))
//select和pselect的fd_set结构体
typedef struct
{
    //成员就是一个长整型的数组，用来实现位图
    __fd_mask __fds_bits[__FD_SETSIZE / __NFDBITS];
} fd_set;

// fd_set里面文件描述符的数量，可以使用ulimit -n进行查看
#define FD_SETSIZE __FD_SETSIZE

```

- maxfd为什么是最大描述符加1呢？
当传入fdmax的时候，select会监听0~fdmax-1的文件描述符

4.4.3 select的退出机制

- 当管道的写端先关闭的时候，读端使用read的时候会返回0（相当于发送了一个EOF），操作系统会将管道的状态设置为可读，可读的状态会影响到select函数，导致select函数不会阻塞，进入死循环。下面是一个简单的例子来说明写端关闭的情况

```
//将写端的程序修改为如此
#include <func.h>
int main(int argc, char *argv[])
{
    ARGS_CHECK(argc,2);
    int fdw = open(argv[1],O_WRONLY);
    ERROR_CHECK(fdw,-1,"open");
    printf("fdw = %d\n",fdw);
    close(fdw);//这里将写端直接关闭
    sleep(10);//然后睡眠10s
    return 0;
}
```

- 当管道的读端先关闭的时候，写端会直接崩溃（这种情况的处理方法要等到网络编程阶段才会介绍）
- 为了避免死循环，需要在读端对退出的情况进行兼容处理，就是当read的返回值为0的时候，就退出程序

```
...
    if(FD_ISSET(STDIN_FILENO, &rdset))
    {
        memset(buf,0,sizeof(buf));
        read_ret = read(STDIN_FILENO, buf, sizeof(buf));
        if(read_ret == 0)
        {
            printf("chat is broken!\n");
            break;
        }
        write(fdw, buf, strlen(buf)-1);
    }
    if(FD_ISSET(fdr, &rdset))
    {
        memset(buf,0,sizeof(buf));
        read_ret = read(fdr, buf, sizeof(buf));
        if(read_ret == 0)
        {
            printf("chat is broken!\n");
            break;
        }
        printf("buf = %s\n", buf);
    }
    ...
```

```
#使用ctrl+c终止程序会导致程序的返回值不为0
#可以改用ctrl+d来终止stdin（相当于输入了EOF）
#$?代表了上个执行程序的返回值
$echo $?
```

4.4.4 select函数的超时处理

- 使用timeval结构体可以设置超时时间。传入select函数中的timeout参数是一个timeval结构体指针，timeval结构体的定义如下：

```
struct timeval
```



```

{
    long tv_sec;//秒
    long tv_usec;//微秒
};
//用法
...
struct timeval timeout;
while(1)
{
    bzero(&timeout, sizeof(timeout));
    timeout.tv_sec = 3;
    ret = select(fdr+1, &rdset, NULL, NULL, &timeout);
    if(ret > 0)
    {
        ...
    }
    else
    {
        printf("time out!\n");
    }
}
}

```

- 使用的超时判断的时候要注意，每次调用select之前需要重新为timeout赋值，因为调用select会修改timeout里面的内容

4.4.5 写集合的原理

- 写阻塞和写就绪：当管道的写端向管道中写入数据达到上限以后，后续的写入操作将会导致进程进入阻塞态，称为写阻塞现象
- 提问：如何实现一个写阻塞？如何检查写阻塞的中断问题？
- 处于写阻塞状态以后，当管道中的数据被读端读取以后，写端就可以恢复写入操作，称为写就绪
- 类似于读文件描述符集合，select也可以设置专门的写文件描述符集合，select可以监听处于写阻塞状态下的文件，一旦文件转为写就绪，就可以将进程转换为就绪

```

#include <func.h>
int main(int argc, char* argv[])
{
    ARGS_CHECK(argc, 2);
    int fdr = open(argv[1], O_RDWR);
    int fdw = open(argv[1], O_RDWR); //可以一次性打开管道的读写端
    fd_set rdset, wrset;
    int ret;
    char buf[128];
    while(1)
    {
        FD_ZERO(&rdset);
        FD_ZERO(&wrset);
        FD_SET(fdr, &rdset);
        FD_SET(fdw, &wrset);
        ret = select(fdw+1, &rdset, &wrset, NULL, NULL);
        if(FD_ISSET(fdr, &rdset))
        {
            bzero(buf, sizeof(buf));
            read(fdr, buf, sizeof(buf));
            puts(buf);
        }
    }
}

```

```

        usleep(250000);
    }
    if(FD_ISSET(fdw, &wrset))
    {
        write(fdw, "helloworld", 10);
        usleep(500000);
    }
}
}

```

4.5 杂项

- 安装vimplus:

1. 安装YouCompleteMe

```
$sudo apt install vim-youcompleteme
```

2. 安装vimplus剩余插件

```

$sudo apt install git
$cd ~
$git clone https://gitee.com/chxuan/vimplus.git ~/.vimplus
$cd .vimplus
$./install.sh
# 如果没有自动补全功能
$vim-addons remove youcompleteme
$vim-addons install youcompleteme

```

3. 在安装了vimplus以后修改snippet.c可设置默认的内容

Select底层原理

select的流程.

①. fdset从用户态拷贝到内核态

② 内核轮询 (poll)

0 ~ nfds-1

③ 发现了监听的资源.就绪
放回fd-set, 返回

Select缺陷

① 位置是靠数组实现, 改变长度重新编译

② 大量的内核态 用户态数据拷贝

③ 监听和就绪 耦合

④ 就绪集合处理性能低 (海量连接, 少量就绪).